NUST
NATIONAL UNIVERSITY
OF SCIENCES & TECHNOLOGY

Project Report

Table Tennis Ball Launcher & Practice Robot

Course: EE-222 Microprocessor Systems

Lab Instructor: Engineer Adnan Aslam

Team Members:

Member Name	Registration Number
Ammar Bin Jalal	502254
Alina Riaz	506611
Muhammad Ahmed Saeed	501801
Muhammad Mutaal Khan	522455

Date: May 12, 2026

Abstract

This project includes the design and implementation of an automated Table Tennis Ball Launcher. The primary problem solved is providing table tennis players with a reliable, repetitive training mechanism to build muscle memory without requiring a human partner. We solved this by developing a microcontroller-based system that autonomously feeds and shoots table tennis balls with continuous variable oscillation. The system utilizes an ATmega328P microcontroller programmed in C via Atmel Studio. Key components include two TT gear motors driven by an BTS 7960 motor driver to grip and shoot the ball, two high-torque servo motors for feeding and oscillation, and Lithium-ion batteries for power. The major results demonstrate highly accurate autonomous execution and consistent ball delivery, validating the system's effectiveness as a sports training tool.

1. Problem Definition and Analysis

In competitive table tennis, players require repetitive practice against specific types of shots to develop muscle memory and improve their reaction times. Manual feeding by a coach or partner can be inconsistent, fatiguing, and lacks precise repeatability. The complex engineering problem (CEP) is to design a microcontroller-based automated system that can reliably feed, shoot, and continually vary the direction of table tennis balls based on programmed autonomous logic.

This problem matters because effective sports automation democratizes high-level training, allowing solo players to practice efficiently. The project faces several constraints, including limited processing power for simultaneous motor control, the need for exact timing delays to prevent ball jamming, and ensuring robust autonomous operation so the player receives consistent practice from the other side of the table.

2. Literature Review & Theoretical Foundation

An **Embedded System** is a dedicated computer system designed for a specific control function within a larger mechanical or electrical system, often with real-time computing constraints. Unlike general-purpose computers, embedded systems are deeply integrated into the hardware they control, orchestrating physical outputs based on sensor inputs or user commands.

At the core of these systems lie microprocessors or microcontrollers. While a **Microprocessor** contains only the central processing unit (CPU) and requires external components such as RAM, ROM, and I/O ports to function, a **Microcontroller** integrates the CPU, memory, and essential programmable peripherals onto a single integrated circuit. This integration makes microcontrollers significantly more compact, power-efficient, and cost-effective for dedicated robotics tasks.

For this project, the **ATmega328P** is widely used and was selected due to its optimal balance of processing power (16MHz) and accessibility. It provides built-in hardware capabilities such as hardware timers (ideal for PWM generation), making it highly suitable for low-level register manipulation in C without overwhelming complexity. For this project, the firmware was developed using Atmel Studio and embedded C language to allow for this precise low-level control. Relevant theoretical concepts utilized include:

- **H-Bridge Motor Control:** To shoot the ball, two DC motors must spin in opposite

directions simultaneously. The driver uses an H-Bridge topology to control the direction and state of these high-current motors using simple GPIO logic.

3. Design and Methodology

3.1 Block Diagram

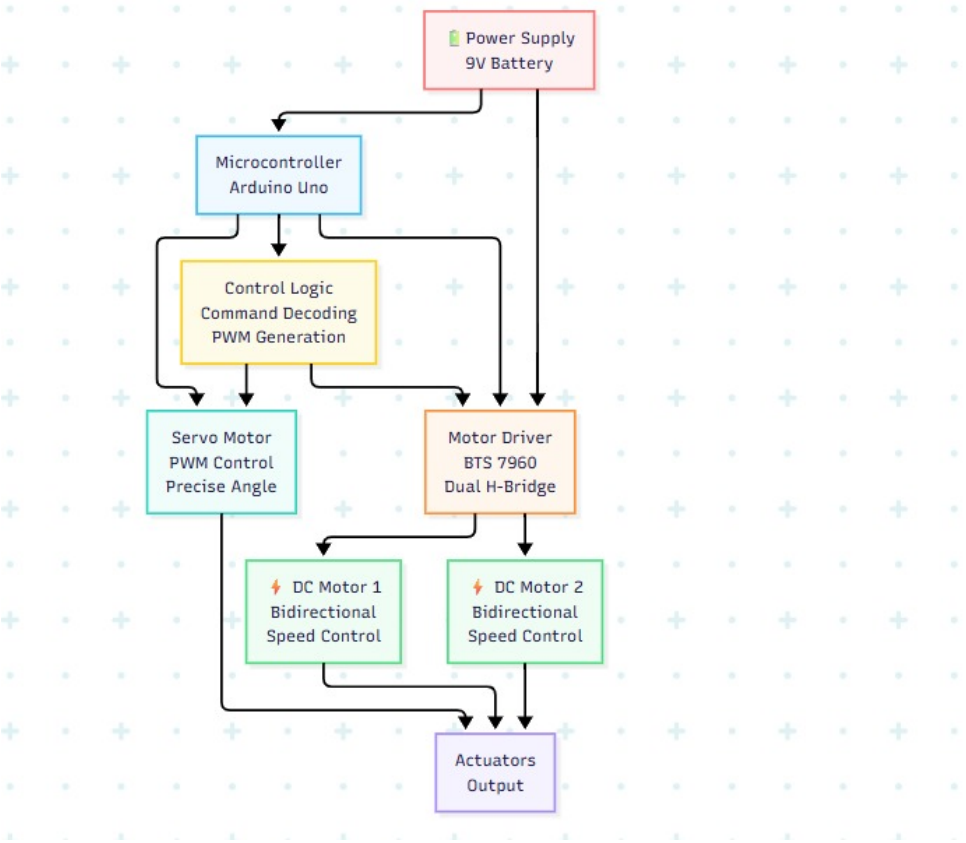

Figure 1: Device → ATmega328P → Motor Driver / Servos → Motors

3.2 Components Used & Justification

Component	Purpose	Why Used
ATmega328P (Arduino UNO)	Controller	Low cost, register-level access via Atmel Studio
2 TT Motors	Output	Spin in opposite directions to grip/shoot the ball
BTS 7960 Motor Driver	Output	Safe handling of high current for TT motors
2 High Torque Servos	Actuation	Controls the ball feeder mechanism and base oscillation
Lithium-ion Batteries	Power	Stable, high-discharge energy for motors

Table 1: Hardware Components and Justification

4. Code Snippet

The following C snippet demonstrates the configuration and logic used in the project:

Listing 1: Main Controller Logic

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>

// Motor Pins: D5 (PD5) and D6 (PD6)
// Servo Pin: D11 (PB3)
void init_pwm() {
        // --- Setup Motor PWM (Timer 0) ---
        // Fast PWM, Non-inverting mode on OC0B (D5) and OC0A (D6)
        TCCR0A = (1 << COM0A1) | (1 << COM0B1) | (1 << WGM01) | (1 <<
            WGM00);
        TCCR0B = (1 << CS01) | (1 << CS00); // Prescaler 64

        // --- Setup Servo PWM (Timer 2) ---
        // Using Timer 2 for D11 (OC2A) to keep it simple
        // Fast PWM, Prescaler 1024 (approx 61Hz, close enough for
           most servos)
        TCCR2A = (1 << COM2A1) | (1 << WGM21) | (1 << WGM20);
        TCCR2B = (1 << CS22) | (1 << CS21) | (1 << CS20);

        // Set Data Direction Registers to Output
        DDRD |= (1 << DDD5) | (1 << DDD6); // Motor Pins D5, D6
        DDRB |= (1 << DDB3);               // Servo Pin D11
}
// Function to set servo position (mapping 0-180 degrees to pulse
   width)
// This is a rough estimation for 50-60Hz PWM
void set_servo_pos(int degrees) {
        // Servo pulse width is usually 1ms to 2ms.
        // On Timer 2 with 1024 prescaler: 0 deg ~ 12, 180 deg ~ 36
        uint8_t ocr_value = 12 + (degrees * 24 / 180);
        OCR2A = ocr_value;}
int main(void) {
        init_pwm();
        // Initial Safety: Motor Off
        OCR0A = 0;
        OCR0B = 0;
        _delay_ms(500);

        // Start Motor: RPWM (D5) = 150, LPWM (D6) = 0
        OCR0B = 150;
        OCR0A = 0;
        while (1) {
                // 1. Sweep 0 to 180 degrees
                for (int pos = 0; pos <= 180; pos++) {
                        set_servo_pos(pos);
                        _delay_ms(30); // Slow speed
                }
                // 2. Stop for 1 second
                _delay_ms(1000);
                // 3. Sweep 180 to 0 degrees
```

```
        for (int pos = 180; pos >= 0; pos--) {
            set_servo_pos(pos);
            _delay_ms(30); // Slow speed
        }
        // 4. Stop for 1 second
        _delay_ms(1000);}}
```

5. Flowchart

The following flowchart illustrates the autonomous control logic of the ATmega328P. The system is designed to initialize hardware, safely start the launch motors, and then continually sweep the servo mechanism to automatically vary the shot placement.

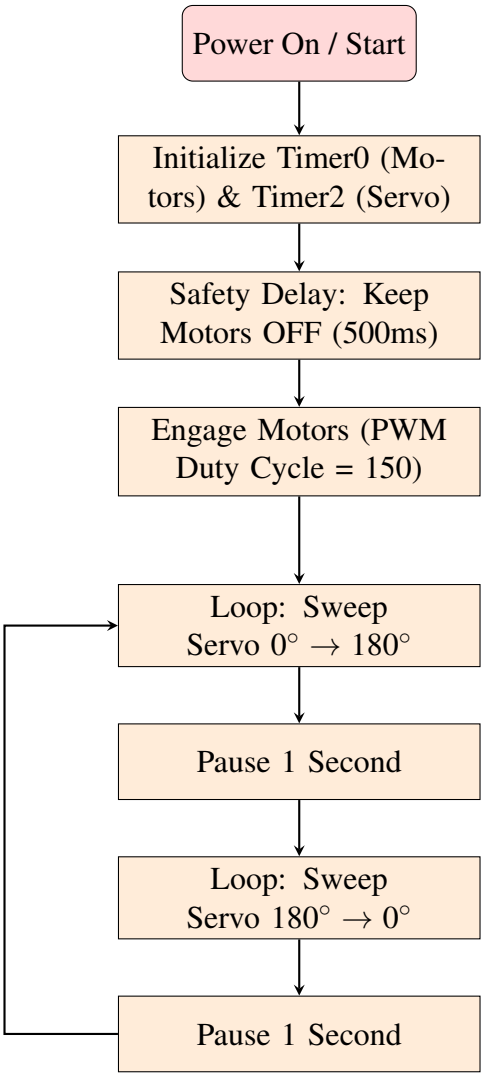

Figure 2: Autonomous System Logic Flowchart

6. Simulation, Modeling, and Implementation

The circuit was fully modeled and simulated using Proteus software prior to hardware implementation. The simulation verified the microcontroller's autonomous PWM generation for both the DC motors and the servo sweeping logic.

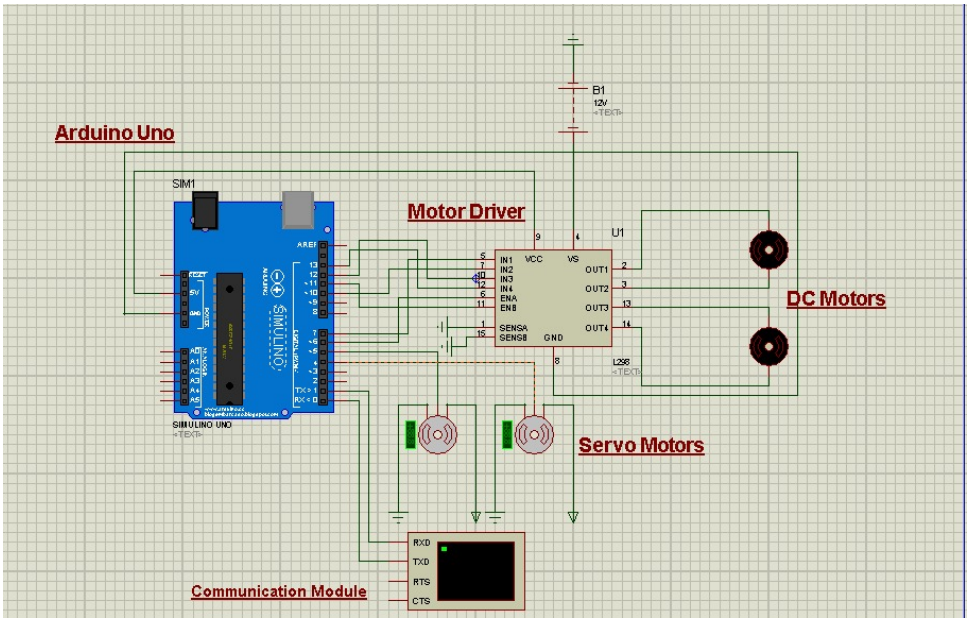

Figure 3: Proteus Simulation showing ATmega328P, Motor Driver, and motors.

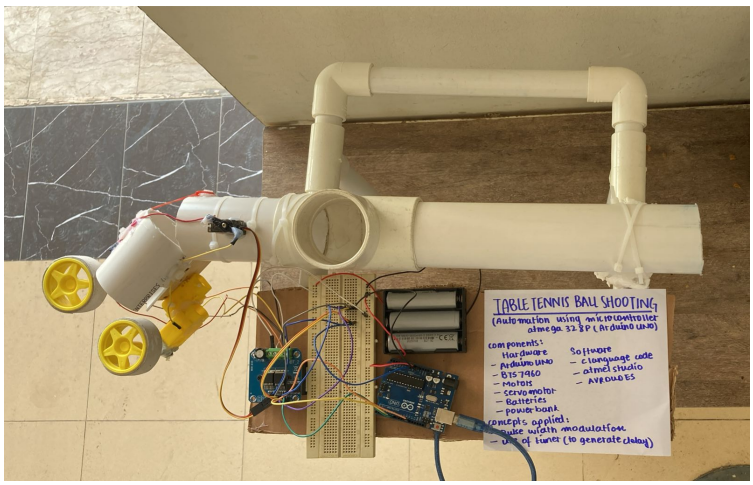

Figure 4: Hardware Design

The transition from simulation to hardware required adjustments, particularly in managing the power draw of the physical TT motors, which required the Lithium-ion battery packs to bypass the microcontroller's onboard voltage regulator.

7. Results, Testing, and Validation

Comprehensive test cases were designed to validate the robot's autonomous performance configured directly through Atmel Studio:

- **Test Case 1 (Motor Initialization and Configuration):** Expected the DC motors to initialize safely and then spin continuously at the programmed PWM duty cycle. Actual result: The motors engaged successfully and maintained a consistent high-speed rotation after the initial 500ms safety delay.
- **Test Case 2 (Autonomous Servo Sweeping):** Expected the servo to continuously sweep from 0° to 180° and back, pausing for 1 second between sweeps, programmed purely via C logic. Actual result: Smooth mechanical oscillation achieved with exact timing, ensuring varied ball direction.
- **Test Case 3 (Ball Launch Range):** Expected the firing mechanism to securely grip and launch the table tennis ball. Actual result: The system successfully grabbed the fed balls and consistently launched them, reaching a fair distance suitable for actual player practice across a standard table.

8. Technical Analysis

The software implementation required precise mathematical calculations to bridge theoretical logic with hardware execution.

8.1 PWM Frequency and Duty Cycle Calculation

The ATmega328P operates at a system clock (F_{CPU}) of 16 MHz. To control the DC motors efficiently, we use Fast PWM mode with an 8-bit timer (256 steps) and a prescaler of 64.

PWM Frequency Calculation:

$$F_{PWM} = \frac{F_{CPU}}{\text{Prescaler} \times 256} \tag{1}$$

$$F_{PWM} = \frac{16,000,000}{64 \times 256} = 976.56 \text{ Hz} \tag{2}$$

A frequency of ≈ 976 Hz is optimal for small DC motors; it is fast enough to ensure smooth continuous rotation without losing torque, and slow enough to prevent excessive switching heat in the motor driver.

Duty Cycle Calculation: In 8-bit Fast PWM, the timer counts from 0 to 255. The Output Compare Register (`OCR0A` or `OCR1A`) determines the duty cycle (the percentage of time the signal is HIGH).

$$\text{Duty Cycle } (\%) = \left(\frac{\texttt{OCRnx}}{255} \right) \times 100 \tag{3}$$

For example, setting `OCR0A = 127` yields a 50% duty cycle ($\approx 2.5V$ average output), driving the motor at exactly half speed. Setting `OCR0B = 150` yields a roughly 58% duty cycle.

9. Innovation, Integration, and Complexity

This project showcases high engineering complexity by bypassing standard Arduino libraries and manipulating hardware registers directly in C. The integration of high-current DC motor logic (H-bridge phase manipulation) alongside precise microsecond-level timing delays for autonomous servo articulation demonstrates a sophisticated mastery of embedded control.

10. Teamwork and Individual Contribution

Member	**Contribution**
Alina Riaz	C Coding and Presentation Preparation
Muhammad Mutaal Khan	Proteus Simulation and Technical Documentation
Ahmed Saeed and Ammar Bin Jalal	Hardware Assembly (Motors, Driver, Frame) and Testing

Table 2: Individual Contributions

11. Professionalism and Ethical Considerations

The physical design prioritizes user safety by completely isolating the high-discharge Lithium-ion motor circuit from the logic circuit using the driver's internal topology. From a professional standpoint, the code is structured modularly with clear documentation, ensuring future students or hobbyists can safely replicate or iterate upon the technology.

12. Conclusion

This project enhanced our understanding of microprocessor systems by providing hands-on experience with the ATmega328P, embedded C programming, and hardware integration. We successfully built an automated table tennis robot, learning how to design real-time systems, debug hardware/software issues, and apply theoretical concepts such as autonomous PWM generation, H-bridge control logic, and microsecond timing in practical applications. The course EE-222 Microprocessor Systems, successfully helped us bridge the gap between theory and real-world engineering problems.

References

[1] Microchip Technology. *ATmega328P Datasheet*.

[2] Mazidi, M. A. *AVR Microcontroller and Embedded Systems*. Pearson.